



PROJECT REPORT

FALL SEMESTER 2026-27

EXPENSE TRACKING SYSTEM

Python-Based Menu-Driven Application

Name:	Janvi Raghvani
Registration Number:	26MIM10243
Project Type:	Console-based Python Application
Submitted To:	Mr. Kannaiya Rajan Sir
Data Storage:	CSV File (expenses.csv)

1. Problem Statement

Keeping track of daily expenses manually can become confusing, especially when there are many small transactions. It can be difficult to remember where the money was spent and how much was spent in total.

To solve this problem, the **Expense Tracking System** was developed as a simple Python-based console application. It allows users to add, view, summarize, and delete their expense records. The information is stored in a CSV file, so the records are not lost when the program is closed and can be accessed again when the program is started.

The application uses a simple menu with five options:

- Add an expense
- View all expenses
- View category-wise spending summary
- Delete an expense
- Exit the program

2. Objectives

The main objectives of this project are:

- To create a simple and easy-to-use expense tracking system using Python.
- To allow users to record their daily expenses along with the amount, category, note, and date.
- To check important inputs such as the amount and date before saving them.
- To store expense records permanently using a CSV file.
- To display all saved expenses along with the total amount spent.
- To show how much money is spent in each category.
- To allow users to remove an expense record when required.
- To apply basic Python concepts such as functions, loops, conditional statements, file handling, CSV handling, and exception handling in a practical project.

3. Functional Requirements

FR1 – Add Expense

The system should allow the user to enter the expense amount, category, optional note, and date. The amount entered must be greater than zero.

FR2 – Date Validation

The user can enter the date in **YYYY-MM-DD** format. If no date is entered, the system automatically uses the current date.

FR3 – Save Data

After an expense is entered correctly, the system saves the information in the expenses.csv file.

FR4 – Load Data

Whenever the program starts, it checks for the existing CSV file and loads the previously saved expense records.

FR5 – View Expenses

The system displays all saved expenses along with their number, date, amount, category, and note. It also shows the total amount spent.

FR6 – Category Summary

The system calculates the total amount spent in each category and shows the percentage contribution of every category to the overall spending.

FR7 – Delete Expense

The user can select an expense using its displayed number and remove it from the records.

FR8 – Menu Navigation

The main menu keeps appearing after every operation until the user chooses the **Exit** option.

4. Non-Functional Requirements

Usability

The application is designed to be simple and beginner-friendly. Users can operate it by selecting numbered options from the menu.

Reliability

The program handles incorrect inputs such as invalid numbers, zero or negative amounts, incorrect dates, and invalid menu choices without suddenly stopping.

Performance

For normal personal expense records, the program should load, save, display, and summarize the data quickly.

Maintainability

The code is divided into separate functions such as `add_expense()`, `view_expenses()`, `show_summary()`, and `delete_expense()`. This makes the program easier to understand and modify.

Portability

The application can run in a normal Python environment and does not require a separate database server or external application.

Data Persistence

The expense records remain available even after the program is closed because they are stored in the CSV file.

5. System Architecture

The system follows a simple three-part structure. First, the user interacts with the application through the console menu. The Python program then processes the selected option and either reads data from or writes data to the CSV file.

System Flow:

User / Console Interface → Python Expense Tracker Application → CSV Storage (expenses.csv)
The Python application contains separate functions for different operations such as adding, viewing, summarizing, and deleting expenses. The CSV file is used to keep the data stored between different program executions.

6. Major System Components

Main Controller

The `main()` function controls the overall program. It loads the existing records, displays the menu, takes the user's choice, and calls the required function.

Expense Input Module

The `add_expense()` function collects the amount, category, note, and date from the user. It checks whether the entered information is valid and then saves the new expense.

Expense Display Module

The `view_expenses()` function displays all stored expenses and calculates the total amount spent.

Summary Module

The `show_summary()` function groups expenses according to their categories and calculates the percentage spent on each category.

Deletion Module

The `delete_expense()` function displays the saved expenses, allows the user to select one, removes it, and saves the updated records.

CSV Storage

The `load_expenses()` function reads existing records from the CSV file, while `save_expenses()` stores the current expense list back into the file.

7. Input and Data Format

Each expense record contains four main values:

- **Date** – The date on which the expense occurred.
- **Amount** – The amount of money spent.
- **Category** – The type of expense, such as Food.
- **Note** – An optional description of the expense.

These values are stored together as one row in the CSV file. When the records are loaded, the amount is converted into a floating-point value so that calculations can be performed easily.

Example:

Date	2026-09-24
Amount	250.00
Category	Food
Note	Lunch

8. Validation and Error Handling

The program checks user input at different stages to avoid incorrect data.

- If the entered amount is not a number, an error message is displayed and the user is returned to the menu.
- If the amount is zero or negative, the system does not accept the expense.
- If the user enters a date in an incorrect format instead of YYYY-MM-DD, the expense is rejected.
- If a non-numeric value is entered while selecting an expense for deletion, the system displays an appropriate error message.
- If the user enters a menu option outside the range of 1 to 5, the program asks the user to enter a valid choice.

9. Conclusion

The **Expense Tracking System** is a simple Python project developed to solve a common day-to-day problem: keeping track of personal expenses.

The application provides the basic features needed for expense management, including adding new expenses, viewing saved records, checking category-wise spending, and deleting unwanted records. Using a CSV file makes the project lightweight while still allowing the data to remain saved between different program runs.

This project also provides practical experience with important Python concepts such as functions, loops, conditional statements, exception handling, date validation, and file handling. Overall, it shows how basic programming concepts can be combined to create a useful real-world application.

10. Future Enhancements

The project can be improved further by adding features such as:

- Weekly and monthly expense reports.
- A budget-setting option with alerts when the spending limit is crossed.
- Search and filtering options based on date or category.
- Graphs and charts for better visualization of spending.
- An option to edit or update existing expenses.
- Replacing CSV storage with SQLite or another database for handling larger amounts of data.
- A graphical user interface (GUI) to make the application more interactive and user-friendly.

THANK YOU!

JANVI RAGHWANI
26MIM10243